



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Izveštaj iz predmeta
Projektovanje elektronskih
sistema

Sistem za automatsko zalivanje bašta

STUDENTI:

MARKO POPOVIĆ 19323

PETAR RISTIĆ 18881

PROFESOR:

PROF. DR BORISAV JOVANOVIĆ

Contents

1	Opis zadatka	2
2	Funkcionalna i nefunkcionalna specifikacija	3
3	Arhitektura sistema	4
4	Realizacija komunikacionog protokola	5
4.1	Komunikacija između Master i Slev automata	5
4.2	Komunikacija između Master automata i Centralnog Računarskog sistema	6
5	Hardverska Realizacija automata	7
5.1	Šema Master automata	7
5.2	Prikaz sadržaja displeja Master automata	9
5.3	Šema Slev automata	10
5.4	Prikaz sadržaja displeja Slev automata	12
6	Realizacija firmvera Master automata	12
6.1	Detaljan opis najvažnijih funkcija sa segmentima koda	16
7	Realizacija firmvera Slev automata	24
7.1	Definicija pinova i deklaracija promenljivih	24
7.2	Korišćene funkcije	26
7.3	Detaljan opis najvažnijih funkcija sa segmentima koda	27
8	Postupak verifikacije i vrednovanja koda	34
8.1	Rezultat rada sistema	34
8.2	Stepen ispunjenosti korisničkih zahteva	34
9	Detaljno uputstvo za upotrebu	35
10	Appendix	36

1 Opis zadatka

Projektovati sistem za automatizovano zalivanje biljaka u baštama. Sistem sadrži sledeće funkcionalne celine: uređaje za zalivanje sa pumpom (do 16 Slejv uređaja) koji se instaliraju u baštama; jedan Master automat i centralni računarski sistem. Slejv uređaji su preko RS-232 povezani sa Master automatom koji prikuplja informacije sa Slejv uređaja. Master automat je dalje povezan preko ETHERNET veze sa centralnim računarom.

Uređaj za zalivanje sa pumpom omogućava zalivanje biljaka po unapred definisanom programu rada. Zalivanje se odvija tako što u definisanom vremenskom trenutku početka zalivanja uređaj uključuje pumpu, a isključuje je posle određenog vremena. Vremena uključivanja pumpe i vremensko trajanje zalivanja definisani su programom rada. Postoji 16 različitih programa rada koji se zadaju sa centralnog računara. Pošto se smatra da je baštu dovoljno zalivati jednom dnevno, u svakom programu definisano je po jedno vreme uključivanja pumpe, vreme trajanja zalivanja (u sekundama). Uređaj za zalivanje poseduje taster za manuelno pokretanje/zaustavljanje procesa zalivanja. Kada se zalivanje pokrene manuelno, isključuje se ili manuelno ili automatski posle isteklog vremena trajanja zalivanja definisanog u tekućem programu rada. Manuelno zaustavljanje moguće je i kada zalivanje otpočne automatski. Protok vode kroz pumpu prati se očitavanjem senzora protoka. Ukoliko je protok izvan unapred zadatog opsega (minimalni i maksimalni protok) aktivira se zvučni alarm. Uređaj za zalivanje poseduje LCD na kome se prikazuju osnovni podaci o tekućem programu rada, statusu sistema, tekućem vremenu. Ukoliko je započeto zalivanje na LCD-u se prikazuje i koliko vremena je preostalo do isključivanja pumpe.

Centralni računar treba da omogućiti:

- Očitavanje statusa uređaja za zalivanje (uključen/isključen, start/stop zalivanja, alarm...) svake sekunde.
- Definisanje svih 16 programa rada ili definisanje pojedinih vremenskih parametara (vreme uključivanja, trajanje zalivanja) za dati program rada
- Daljinsko podešavanje maksimalnog i minimalnog dozvoljenog protoka vode kroz pumpu
- Indikaciju alarma.
- Manuelno daljinsko uključivanje i isključivanje sistema, kao i daljinsko pokretanje i zaustavljanje procesa zalivanja

2 Funkcionalna i nefunkcionalna specifikacija

Funkcionalna specifikacija :

- Sistem se sastoji iz jednog Master i do 16 Slejv automata
- Master automat predstavlja centralni automat koji prikuplja informacije od svih Slejv automata i upravlja njima
- Master automatom se upravlja putem Centralnog Računarskog Sistema, a samim tim i Slejv automatima
- Slejv automati su kontroleri zalivanja koji se nalaze na svakoj bašti i upravljaju protokom i trajanju programa
- Master automat čuva statusne informacije od svih prisutnih Slejvova
- Master vrši kalibraciju Sata Realnog Vremena svih Slejv automata na zahtev CRS-a
- CRS ima funkciju prikazivanja statusa bašte, podešavanja Sata Realnog Vremena i programiranje programa rada
- Master automat čuva podatke o 16 programa zalivanja, koje dodeljuje Slejvovima na zahtev CRS-a
- Slejv automat odgovara periodično o svom trenutnom stanju
- Vrš se prikaz osnovnih podataka na Master ekranu

Nefunkcionalna specifikacija :

- Uređaji moraju biti otporni na vlagu, vodu i prašinu, biti dobro izolovani, zbog svog rada u izloženim sredinama
- Takođe moraju imati malu potrošnju, i visoku pouzdanost

3 Arhitektura sistema

Arhitekturu sistema čine tri osnovne komponente sistema:

- Centralni Računarski Sistem (CRS)
- Master Automat
- Slave automat

CRS je sistem koji služi za upravljanje Master automatom i samim tim i Slejv automatima. Povezan je sa Masterom automatom putem Ethernet veze, statičkom IP adresom, na njihovoj lokalnoj mreži. Pristup Master automatu vrši se putem HTTP protokola, kucanjem statičke adrese direktno u browser.

Master automat upravlja Slejv automatima, kojih ima maksimalno 16. Vršiti se pozivka svakog Slejva na 125ms, pri kojoj se prikupljaju osnovni podaci o statusu svakog Slejva po protokolu koji će kasnije biti opisan. Automati su u slučaju kratkog udaljenosti međusobno povezani putem RS-232 protokola, dok je u suprotnom korišćen RS-485 protokol.

Slejv automat se nalazi kod pumpe svake bašte u sistemu, i upravlja protokom i vremenom operacije pumpe po prethodno definisanom programu koji mu je Master automat dodelio. Takođe ima funkciju alarma u slučaju prevelikog ili premalog protoka vode kroz sistem za zalivanje.

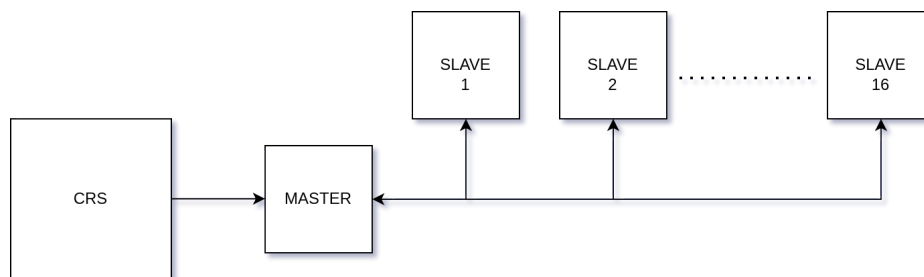


Figure 3.1: Dijagram arhitekture sistema

CRS ima jednosmernu komunikaciju ka Masteru, dok Master i Slejv automat realizuju obostranu.

4 Realizacija komunikacionog protokola

4.1 Komunikacija između Master i Slevj automata

Komunikacija između Master i Slevj automata realizovana je preko UART modula mikrokontrolera i RS-232 drajvera. RS485 je izabran zato što omogućava rad više uređaja na istoj magistrali i pouzdan diferencijalni prenos na većim rastojanjima. U realizovanom sistemu komunikacija radi na bitskoj brzini od 19200 bps, a smer rada RS485 drajvera određuje signal DR. Kada automat šalje podatke, DR se postavlja na logičku jedinicu, a nakon slanja se vraća na nulu da bi automat ponovo slušao magistralu.

Master periodično proziva Slevj automate. Svaki Slevj ima jedinstveni identifikacioni broj od 0 do 15, koji se u firmveru Slevja čita sa donja četiri bita porta D:

$$\text{GARDEN_ID} = \text{PORTD} \& 0x0F$$

Na taj način jedna ista verzija programa može da se koristi na svim Slevj uređajima, a adresa uređaja se podešava prekidačima na razvojnoj ploči.

U implementiranom protokolu koriste se tri osnovna tipa poruka:

- **Prozivka statusa** – Master šalje jedan bajt oblika $0x20 \mid \text{ID}$. Slevj čiji se ID poklapa sa donja četiri bita poruke postavlja `CallFlag` i vraća odgovor od dva bajta.
- **Podešavanje sata realnog vremena** – Master šalje broadcast bajt $0x7F$, zatim tri bajta: sat, minut i sekund. Slevj primljene decimalne vrednosti konvertuje u BCD cifre koje koristi za interni sat.
- **Podešavanje programa zalivanja** – Master šalje bajt $0xA0 \mid \text{ID}$, zatim vreme početka programa i trajanje zalivanja. Slevj čuva primljene vrednosti i koristi ih pri automatskom startovanju pumpe.

Odgovor Slevj automata na prozivku ili uspešno podešavanje sastoji se iz komandnog bajta i statusnog bajta. Komandni bajt ima oblik $0x20 \mid \text{GARDEN_ID}$, čime Master može da proveri od kog Slevja je stigao odgovor. Statusni bajt koristi gornju polovinu bajta za najvažnija stanja uređaja:

Bit	Maska	Značenje
7	0x80	Sistem je uključen
6	0x40	Zalivanje je aktivno
5	0x20	Alarm protoka je aktivan
4	0x10	Zalivanje je pokrenuto manuelno

Preostala četiri bita statusnog bajta ostavljena su slobodna za eventualno proširenje protokola. Statusni bajt formira funkcija `buildStatusByte()`. Ona na osnovu internih promenljivih za uključen sistem, zalivanje, alarm i manuelni režim postavlja odgovarajuće maske.

Poruka za podešavanje programa zalivanja u realizovanom firmveru sadrži četiri korisna bajta:

Redni broj ba- jta	Promenljiva u Slejvu	Značenje
1	<code>Tmp_ProgStartHour</code>	Sat početka zalivanja
2	<code>Tmp_ProgStartMin</code>	Minut početka zalivanja
3	<code>Tmp_time_left_high</code>	Viši deo trajanja zalivanja
4	<code>Tmp_time_left_low</code>	Niži deo trajanja zalivanja

Kada se prijem ove poruke završi, postavlja se `ProgramSetupFlag`. Glavna petlja zatim prebacuje privremene vrednosti u aktivne promenljive programa. Vreme početka se konvertuje u BCD format, jer se u istom formatu vodi interni sat, dok se trajanje zalivanja računa kao:

$$\text{WateringSec} = \text{time_left_high} \cdot 100 + \text{time_left_low}$$

Time se omogućava da Master pošalje program rada, a Slejv da ga izvršava lokalno bez stalnog nadzora centralnog računara.

4.2 Komunikacija između Master automata i Centralnog Računarskog sistema

Komunikacija se odvija putem Ethernet porta, kroz web browser na Centralnom Računarskom Sistemu. Master automat podešen je da ima statičku IPv4 adresu 10.99.21.1, koja je na mrežim Centralnog Računarskog Sistema. Da bi se komuniciralo sa Slejv automatom putem CRS-a, potrebno je dokucati komandu nakon IP adrese. Neke od sledećih komandi su : Gde su:

Komanda	Parametri	Opis
<code>/r</code>	HH MM SS	RTC setup
<code>/b</code>	SlaveID ProgramID	Programiranje date bašte datim programom
<code>/p</code>	ProgramID HH MM SSSS	Podešavanje datog programa
<code>/s</code>	SlaveID	Prikaz trenutnog statusa bašte formatiranog u string

Figure 4.1: Tabela komandi

Parametri	Opis
HH	Sati
MM	Minuti
SS	Sekunde
SlaveID	Identifikacioni broj bašte
ProgramID	Identifikacioni broj programa

Figure 4.2: Tabela parametara

5 Hardverska Realizacija automata

5.1 Šema Master automata

Električna šema Master automata sastoji se od 2x16 LCD-a, RS-232, RS485 i Ethernet interfejsa, pinova i bloka za programiranje mikrokontrolera, mikrokontroler PIC18F2525, bloka za napajanje, četiri tastera povezana na PORTB, svaki sa svojim prekidom, i dve LED diode, od koje se jedna koristi kao znak za prolazak kroz polling svake bašte. Zbog razlike napona komunikacije između Ethernet interfejsa i mikrokontrolera, koristimo level shiftere i buffere (dva redno povezana invertora).

Za naš sistem izabran je PIC18F2525 u svom originalnom DIP28 kućištu. Sa svojih 25Mhz i naponu napajanja od 5V i više je nego dovoljan za zahteve projekta. Ima 48kB ROM memorije, i malo manje od 4kB SRAM memorije, od kojih je u konačnom realizovanom projektu iskorišćeno nešto više od 1kB. Sadrži podršku za RS-232 putem internog oscilatora, Auto-Wake up on start bit, Auto-Baud detect i deseto bitni trinaestokanalni AD konvertor koji sadrži auto akviziciju i konverziju tokom sleep moda procesora. Da bi smo programirali ovaj mikrokontroler koristimo Pic Kit 3 programator.

Na ploči nalazimo i ENC28J60, Ethernet drajver predstavljen u SOIC-28 pakovanju, koji funkcioniše na 25Mhz na naponu od 3.3V.

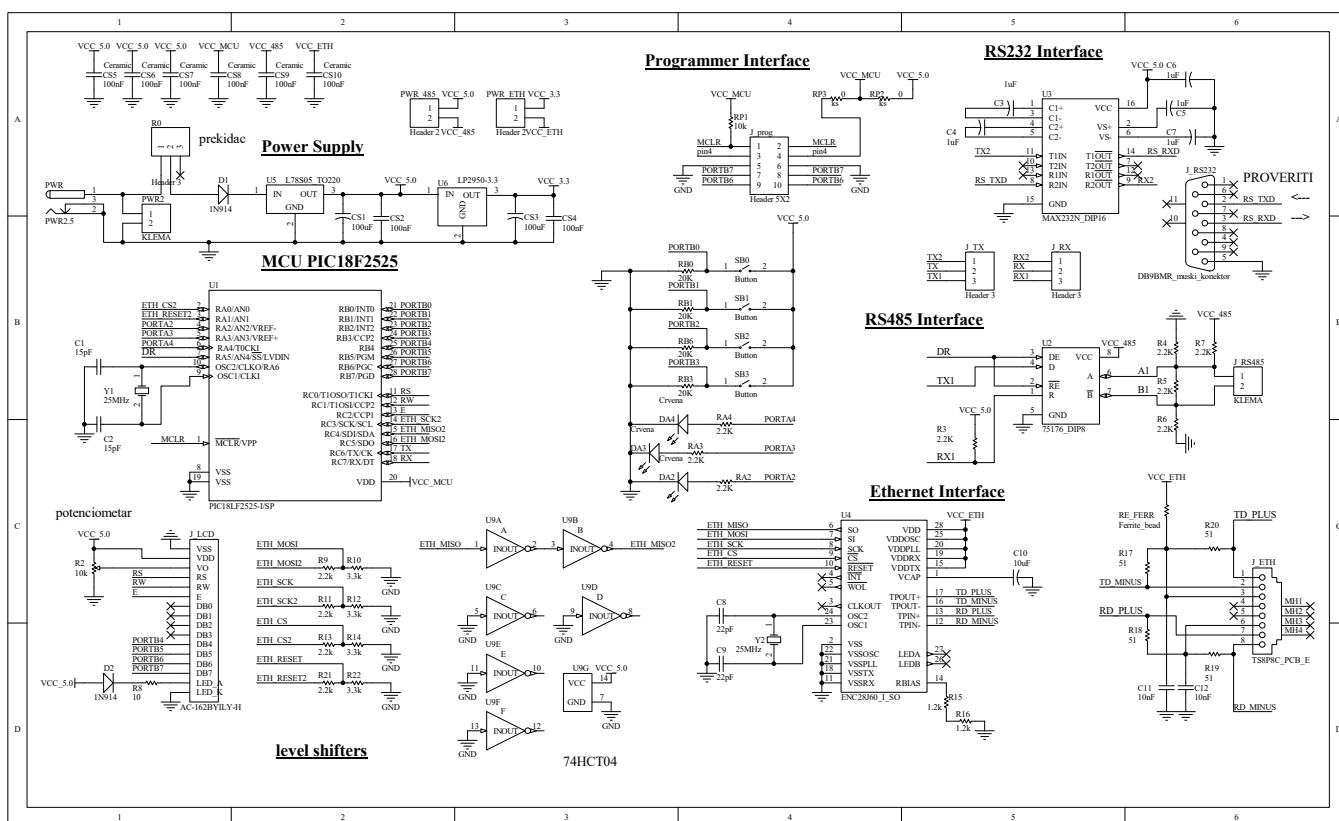


Figure 5.1: Električna šema Master automata

Port	Smer	Opis
PORTA.F5	Izlaz	DR - Write/read kontrola RS485 drajvera SN65176, Postavlja se na 1 pri upisu.
PORTB.F4-F7	Izlaz	Deo magistrale za prenos podataka između LCD-a i PIC-a
PORTC.F0	Izlaz	RS, Data/Instruction select kontrolni signal LCD displeja
PORTC.F1	Izlaz	Read/Write za LCD, 0 je za čitanje
PORTC.F2	Izlaz	Enable signal za LCD
PORTA.F0	Izlaz	Ethernet reset
PORTA.F1	Izlaz	Ethernet chip select
PORTC.F6	Izlaz	TX za UART
PORTC.F7	Izlaz	RX za UART
PORTB.F0	Ulaz	Inkrement dugme
PORTB.F1	Ulaz	Dekrement dugme

Figure 5.2: Tabela portova mikrokontrolera

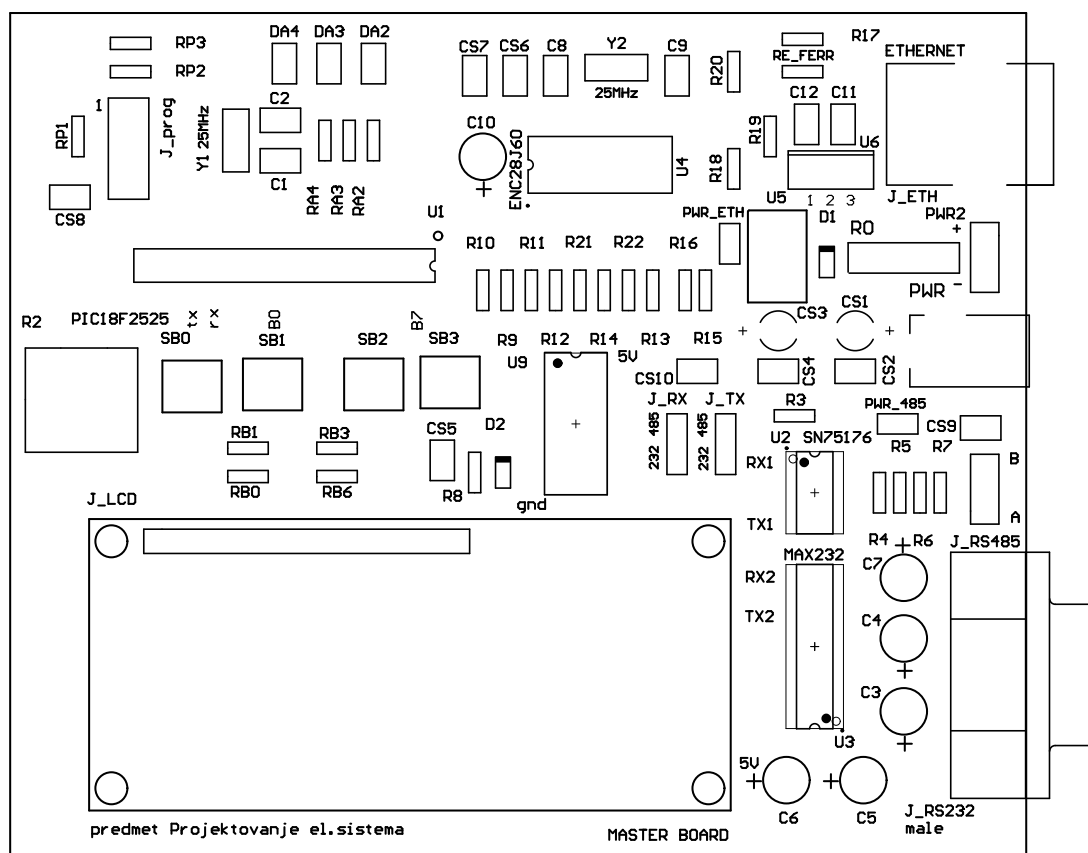


Figure 5.3: Top overlay Master automata

5.2 Prikaz sadržaja displeja Master automata

Master automat sadrži 2x16 LCD displej na kome prikazujemo podatke od značaja, naime programe i statuse bašti. Imamo dvadeset različitih prikaza, od kojih su šestnaest za programe a ostala četiri za stanja Slejv automata. Prikazi se menjaju pritiskom na jedan od dva tastera, Inkrement i Dekrement, povezani na PORTB.F0 i PORTB.F1 respektivno.

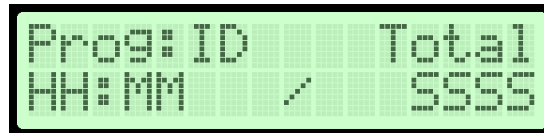


Figure 5.4: Prikaz programa

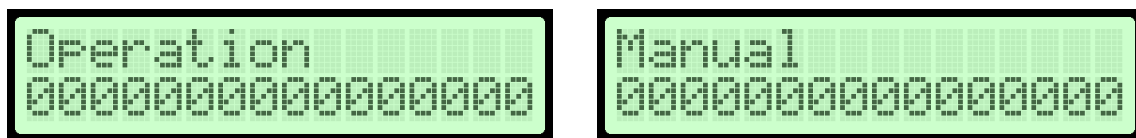


Figure 5.5: Prikaz dostupnih bašti i bašti koje se ručno zalivaju

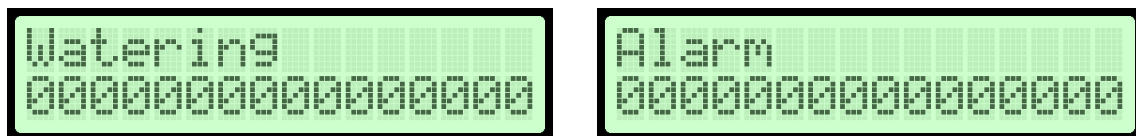


Figure 5.6: Prikaz bašti koje se trenutno zalivaju i čiji je alarm uključen

5.3 Šema Slejv automata

Električna šema Slejv automata sastoji se od razvojne ploče sa mikrokontrolerom PIC16F877A, RS-232/RS485 komunikacionog interfejsa, 16x2 LCD displeja, tastera za manuelno zalivanje, tastera za resetovanje lokalnog stanja, analognog ulaza za senzor protoka i indikacionih izlaza za stanje sistema, rad pumpe i alarm. Slejv automat je postavljen kod pumpe za zalivanje i zadužen je za lokalno upravljanje pumpom, očitavanje senzora protoka, prikaz stanja na displeju i komunikaciju sa Master automatom.

Za Slejv automat izabran je PIC16F877A, jer obezbeđuje dovoljan broj digitalnih ulaza i izlaza, UART komunikaciju, Tajmer 1, A/D konvertor i paralelno povezivanje LCD displeja. Mikrokontroler radi na taktu od 20 MHz i napajanju od 5 V. Kao i kod Master automata, serijska komunikacija koristi UART modul, dok se izbor između RS-232 i RS485 veze vrši kratkospojnicima na razvojnoj ploči. RS-232 je pogodan za razvoj i testiranje sa jednim uređajem na malom rastojanju, dok se u normalnom radu sistema koristi RS485, jer omogućava povezivanje većeg broja Slejv automata na zajedničku magistralu.

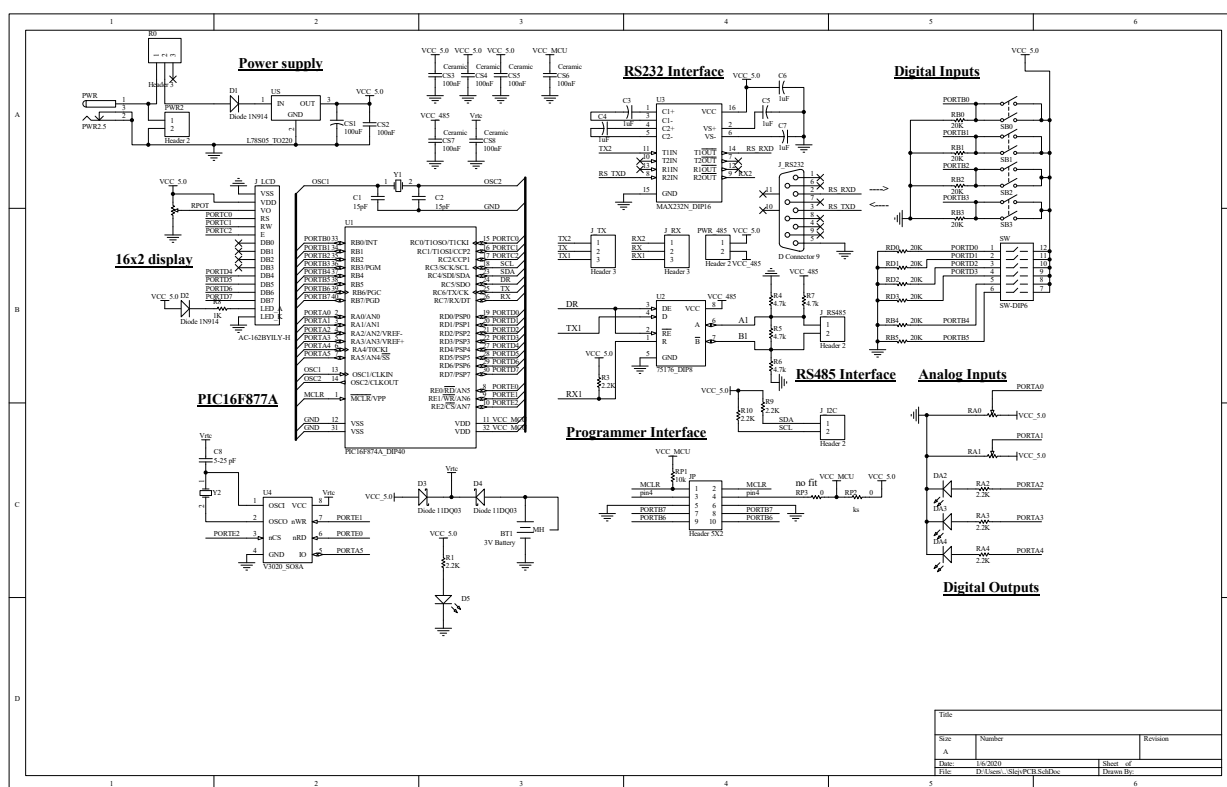


Figure 5.7: Električna šema Slejv automata

Raspored korišćenih pinova u realizovanom firmveru prikazan je u tabeli:

Pin/port	Smer	Namena
PORTB.F0	ulaz	Taster za manuelno pokretanje i zaustavljanje zalivanja
PORTB.F2	ulaz	Taster za resetovanje lokalnog stanja Slejva
PORTA.F0	ulaz	Analogni ulaz A/D konvertora za senzor protoka
PORTA.F2	izlaz	Indikacija da je sistem uključen
PORTA.F3	izlaz	Indikacija rada pumpe, odnosno aktivnog zalivanja
PORTA.F4	izlaz	Indikacija alarma protoka
PORTC.F5	izlaz	DR signal za izbor smera RS485 drajvera
PORTC.F6	izlaz	UART TX linija
PORTC.F7	ulaz	UART RX linija
PORTD.F0-F3	ulaz	Četvorobitni ID Slejv automata
PORTD.F4-F7	izlaz	Četvorobitna magistrala podataka LCD displeja
PORTC.F0	izlaz	RS kontrolni signal LCD displeja </td
PORTC.F2	izlaz	EN kontrolni signal LCD displeja

Figure 5.8: Tabela portova mikrokontrolera Slejv automata

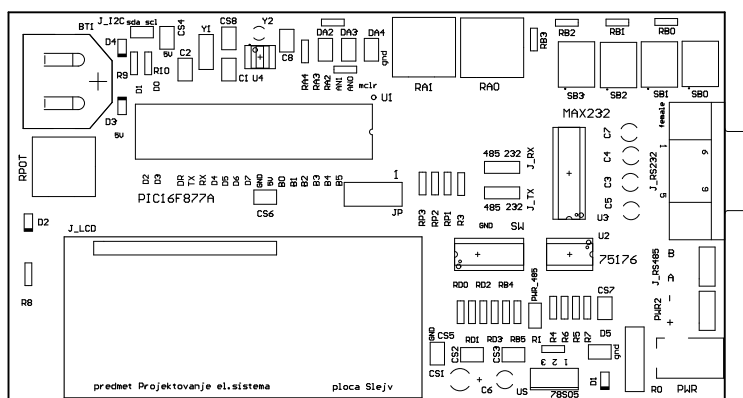


Figure 5.9: Top overlay Slejv automata

5.4 Prikaz sadržaja displeja Slejv automata

Slejv automat sadrži 16x2 LCD displej na kome se prikazuju podaci potrebni za lokalni nadzor rada pumpe. Za razliku od Master automata, koji prikazuje programe i status svih bašti, Slejv prikazuje samo stanje lokalne bašte: trenutno vreme, protok, alarm, manuelni režim, vreme početka programa i preostalo vreme zalivanja.

Prvi red displeja sadrži trenutno vreme i očitanu vrednost protoka:

T HH:MM:SS Fxxx

Drugi red prikazuje da li su aktivni alarm i manuelni režim, vreme početka programa i preostalo vreme zalivanja:

AM S:HH:MM RMM:SS

Karakter A označava aktivan alarm, a karakter M manuelni režim. Kada odgovarajuće stanje nije aktivno, na tom mestu se prikazuje znak /. Oznaka S označava vreme početka zadatog programa zalivanja, dok oznaka R označava preostalo vreme rada pumpe. Vrednost protoka se prikazuje kao trocifren broj iza oznake F.

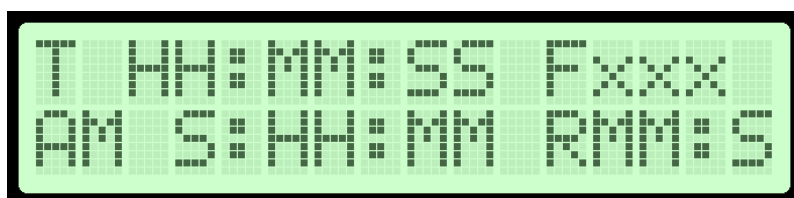


Figure 5.10: Format prikaza Slejv displeja

6 Realizacija firmvera Master automata

Firmver Master automata pisan je u C jeziku i kompajliran na mikroC PRO kompajleru za PIC seriju mikrokontrolera. Kod za firmver se sastoji od funkcija za inicijalizaciju registara mikrokontrolera, inicijalizaciju globalnih promenljivih, funkcija za komunikaciju između automata i centralnog računarskog sistema, globalnog timer interrupta, funkcija za izpis na LCD i main funkcije.

U nastavku su date tabele makroa, pinova, varijabli, struktura i funkcija.

Struktura Mode opisuje program koji zadajemo putem browsera, sadrži sve potrebne elemente za programiranje pumpe bašte.

Struktura Garden sadrži potrebni identifikacioni broj programa, kako bi smo osigurali da svaka bašta može da ima svih 16 postojećih programa. Kako svaki član koristi isti tip podataka, izbegli smo padding, doprineli smo brzini programa i njegovoj čitljivosti i održavanju.

Data je tabela svih globalnih promenljivih korišćenih u Master firmveru.

Figure 6.1: Tabela makroa i pinova

Naziv makroa ili pina	Dodeljena vrednost	Opis
SPI_Ethernet_HALFDUPLEX	0	Komunikacija sa CRS je u punom dupleksu
SPI_Ethernet_FULLDUPLEX	1	Komunikacija sa CRS je u punom dupleksu
DR	PORTA.F5	RS-232 kontrola R/W
ButtonInc	PORTB.F0	Inkrement dugme za promenu prikaza
ButtonDec	PORTB.F1	Dekrement dugme za promenu prikaza

Figure 6.2: Struktura Mode

Naziv člana	Tip	Opis
startHour	unsigned char	Sat početka programa
startMinute	unsigned char	Minut početka programa
durationsH	unsigned char	Gornje dve cifre trajanja programa
durationsL	unsigned char	Donje dve cifre trajanja programa

Figure 6.3: Struktura Garden

Naziv člana	Tip	Opis
modeID	unsigned char	Identifikacioni broj programa dodeljen bašti
gardenSend	unsigned char	Marker za slanje programa određenoj bašti

Figure 6.4: Tabela globalnih promenljivih

Naziv promenljive	Tip	Početna vrednost	Opis
Flag 1	unsigned char	0x00	Flag za 125ms
FlagRTC	unsigned char	0x00	Flag za slanje RTC po- dataka
FlagPoll	unsigned char	0x00	Flag da su svi Slejv atuo- mati prozvani
updateLCDFlag	unsigned char	0x00	Flag za osvežavanje LCD-a
SLAVE_ID	unsigned char	0x00	Jedinstveni identifikacioni broj svakog Slejv automata
ByteID	unsigned char	0x00	Jedinstveni identifikacioni broj svakog dolazećeg bajta
cmd[]	unsigned char	0x00	Komandni bajt svakog Sle- jva
Comm[]	unsigned char	0x00	Niz slejvova koji su se odaz- vali
Status[]	unsigned char	0x00	Status bajt koji svaki Slejv šalje Masteru na prozivku
no_ch	unsigned char	0x00	Brojač karaktera u nizu, može da se koristi i za pozi- cioniranje na početak niza
brojac	unsigned char	0x00	Brojač za tajmer 5x25ms
cntTas	unsigned char	0x00	Brojač za treperenje tastera
cntDisp	unsigned char	0x00	Brojač za prikaze na LCD-u
buffer[250]	unsigned char	0x00	Buffer za smeštanje odgov- ora od Slejvova

Ispod imamo tabelu svih funkcija korišćene za realizaciju rada Master automata. Odmah nakon tabele slede i njihova objašnjenja.

Figure 6.5: Tabela funkcija

Naziv funkcije	Tip	Opis
init()	void	Podešavanje rada pinova mikrokontrolera, konfigurisanje periferija kontrolera, A/D konvertera, prekida, kola tajmera i inicijalizacija UART-a
init_variables	void	Inicijalizacija početnih vrednosti promenljivih
interrupt()	void	Obrada prekida i debounce tastera
unsigned int SPI_Ethernet_UserTCP(unsigned char *remoteHost, unsigned int remotePort, unsigned int localPort, unsigned int reqLength, char *canCloseTCP)	u_int	Obrađuju se zahtevi koji pristižu iz WEB pretraživača i izvršava se slanje podataka preko Etherneta
formBuffer()	void	Formira Niz podataka koji dobije od Slejva
appendBuffer()	void	Dodaje članove u niz
transmit()	void	Slanje podataka preko UARTA
updateLCD()	void	Ispis na LCD ekranu
lcdDisplayProgram(struct Mode Program, unsigned char ID)	void	Formatiranje prikaza programa
lcdDisplayUchar(unsigned char row, unsigned char col, unsigned char value)	void	Formatiranje prikaza unsigned chara
lcdDisplayBit(unsigned char mask)	void	Formatiranje prikaza bitova na ekranu
main()	void	Glavna funkcija programa u kojoj se vrši komunikacija i osvežavanje LCD-a

6.1 Detaljan opis najvažnijih funkcija sa segmentima koda

Funkcija `init_variables` služi za inicijalizaciju varijabli na njihovu podrazumevanu početnu vrednost.

```
void init_variables() {  
  
    no_ch = 0x00;  
    ByteID = 0x00;  
    Flag1 = 0x00;  
    FlagRTC = 0x00;  
    FlagPoll = 0x00;  
    updateLCDFlag = 0x00;  
    btnCnt = 0x00;  
    cntDisp = 0x00;  
    hours = 0x00;  
    minutes = 0x00;  
    seconds = 0x00;  
    SLAVE_ID = 0x0F;  
    for (i = 0; i < 16; i++) {  
        Comm[i] = 0x00;  
        Status1[i] = 0x00;  
        Program[i].startHour = 0x00;  
        Program[i].startMin = 0x00;  
        Program[i].durationsH = 0x00;  
        Program[i].durationsL = 0x00;  
        Garden[i].modeID = 0x00;  
        Garden[i].gardenSend = 0x00;  
    }  
}
```

Funkcija `init` ima zadatak da inicijalizacije portove, registre, UART Baud rate, tajmerski prekid i ethernet.

```
void init() {  
  
    PIR1 = 0b00000000; // flegovi prijema preko UART-a  
    PIE1 = 0b00100001; // dozvola prekida za UART, RCIE, TMR1IE  
    // PIE1.TMR1IE = 1;  
    // PIE1.RC1IE=1;  
  
    T1CON = 0b10110000; // konfiguracija za tajmer1  
    T1CON.TMR1ON = 1;  
    /*16-bit operation  
    preskaler 1:8  
    25MH T0=40ns  
    40ns*4*8=1.28us  
    25ms=25000us=1.28*19531= B5B3 */  
    TMR1L = 0xB5;  
    TMR1H = 0xB3;
```

```

INTCON = 0b01000000; // periferijski interapt
INTCON.GIE = 1;      // globalna dozvola prekida

// svi pinovi na portu A su izlazi (vidi na semi)
TRISA = 0x00;
// obrati paznju, postoje tasteri na portovima RB0 - RB3, to su digitalni
// ulazi
TRISB = 0x0F; // ostali pinovi PORTB su izlazi
TRISC = 0xD0; // 0b11010000; // ovo je OK

PORTA = 0x00;
PORTB = 0x00;
PORTC = 0x00;

ADCON0 = 0x00; // isključujemo A/D konverziju
ADCON1 = 0x0F; // svi digitalni
// PORTA.F0=0;

UART1_Init(UART_BAUD_RATE);

// konfigurisemo brzinu od 19200 i dodatne bitove za serijski prenos
TXSTA.TXEN = 1;
RCSTA.SPEN = 1;
RCSTA.CREN = 1;
SPI1_Init_Advanced(_SPI_MASTER_OSC_DIV64, _SPI_DATA_SAMPLE_MIDDLE,
                  _SPI_CLK_IDLE_LOW, _SPI_LOW_2_HIGH);
SPI_Ethernet_Init(myMacAddr, myIpAddr, SPI_Ethernet_FULLDUPLEX);
Lcd_Init();
Lcd_Cmd(_LCD_CLEAR);
Lcd_Cmd(_LCD_CURSOR_OFF);
// UpdateLCD();
}

```

Funkcije `appendBuffer` i `formBuffer` služe za formiranje nizova koji sadrže odgovore Slevj automata koji će kasnije biti prikazani na CRS, na zahtev klijenta.

```

void appendBuffer(char *p_ch) {
    while ((*p_ch) != 0x00) {
        buffer[no_ch] = *p_ch;
        no_ch++;
        p_ch++;
    }
}

void formBuffer() {
    unsigned char i = 0;
    unsigned char txt[4];
    unsigned char StatusByte = 0x00;
    no_ch = 0x00; // pozicioniranje na pocetak niza

```

```

for (i = 0; i < 16; i++) {
    if (Comm[i] == 1) { // samo slejuovi koji su se odazvali
        appendBuffer("Basta:");
        ByteToStr(i, txt);
        appendBuffer(txt); // append broj baste
        appendBuffer(" ");

        StatusByte = Status1[i];
        // prosto testiranje SWAM nibbla koji slave salje na
        // master i ispisivanje statusa na master racunar
        if (!(StatusByte & STATUS_SYSTEM_BIT)) {
            appendBuffer("OFF\n\n");
        } else if (StatusByte & STATUS_WATER_BIT) {
            if (StatusByte & STATUS_MANUAL_BIT) {
                appendBuffer("Watering(Manual_Mode)\n\n");
                if (StatusByte & STATUS_ALARM_BIT) {
                    appendBuffer("ALARM ON\n\n");
                }
            } else {
                appendBuffer("Watering(Automatic_Mode)\n\n");
                if (StatusByte & STATUS_ALARM_BIT) {
                    appendBuffer("ALARM ON\n\n");
                }
            }
        } else {
            appendBuffer("IDLE\n\n");
        }
        appendBuffer("<br>")
    }
}
buffer[no_ch] = 0x00;
no_ch++; // kraj stringa
}

```

Funkcija transmit ima ulogu slanja bajta putem UART veze, poziva se u main funkciji kako bi se ostvarila komunikacija sa Slejv automatom.

```

void transmit(unsigned char DATA8b) {
    TXREG = DATA8b;
    while (!TXSTA.TRMT)
        ;
}

```

Funkcija interrupt je jedna od najbitnijih funkcija u celom firmveru. Služi za obradu prekida, obradu i debouncing tastera za promenu prikaza na LCD displeju, i komunikaciju između Slejv i Master automata, po protokolu koji je opisan u poglavlju 4.1. Obrada tranzijentnog stanja tastera vrši se u ovoj funkciji po principu brojača koji broji naniže, oba tastera obrađena su na potpuno isti način. Bitovi registara tajmera takođe dobijaju potrebne vrednosti, naime `PIE1.TMR1IE`, čija je namena da dozvoli prekid, ne menja vrednost, dok se `PIR1.TMR1IF` (Timer interrupt flag) stavlja na nizak nivo. Nakon što je prekid obrađen, registri dobijaju vrednosti `TMR1H = 0xB3` i `TMR1L = 0xB5` kako bi smo osigurali da prekid ostane na 25ms.

```
void interrupt() {
    if ((PIE1.TMR1IE == 1) && (PIR1.TMR1IF == 1)) {
        // prekid tajmera 1 - na svakih 25ms PIE1.TMR1IE = 1;
        PIR1.TMR1IF = 0;
        if (brojac == 0x04) { // na svakih 125ms prozivka slejvova
            brojac = 0x00;
            Flag1 = 0x01;
        } else {
            brojac++;
        }
        // 25ms
        if (btnCnt > 0)
            btnCnt--;
        if ((ButtonInc == 1) && (btnCnt == 0))
        {
            btnCnt = 20; // 20*25=500ms
            updateLCDFlag = 1;
            if (cntDisp >= 20) {
                cntDisp = 0;
            } else {
                cntDisp++;
            }
        }
    }

    if ((ButtonDec == 1) && (btnCnt == 0))
    {
        btnCnt = 20; // 20*25=500ms
        updateLCDFlag = 1;
        if (cntDisp == 0) {
            cntDisp = 20;
        } else {
            cntDisp--;
        }
    }

    TMR1L = 0xB5;
    TMR1H = 0xB3; /// reset tajmera
}
```

```

if ((PIE1.RCIE) && (PIR1.RCIF)) { // UART RECIEVE FROM SLAVE
    unsigned char ch;
    PIR1.RCIF = 0; // Recieve flag na 0
    ch = RCREG; // prima se bajt preko UART-a

    switch (ByteID) {
    case BYTE_ID_IDLE:
        break;
    case BYTE_ID_CMD_BYTE:
        if ((ch & CMD_TYPE_MASK) == STATUS_CODE) {
            if ((ch & CMD_ID_MASK) == SLAVE_ID) {
                Comm[SLAVE_ID] = 1;
                ByteID = BYTE_ID_STATUS_BYTE;
            } else {
                ByteID = BYTE_ID_IDLE;
            }
        } else {
            ByteID = BYTE_ID_IDLE;
        }
        break;
    case BYTE_ID_STATUS_BYTE:
        Status1[SLAVE_ID] = ch; // SWAM0000
        ByteID = BYTE_ID_IDLE;
        break;
    default:
        ByteID = BYTE_ID_IDLE;
        break;
    }
}
}

```

Funkcija lcdDisplayUchar je namenska funkcija za prikazivanje unsigned char karaktera na ekran. Funkcija dobije dvocifren broj koji prostim brojanjem naniže odvađa u zasebne dve cifre (ćelije) za prikaz na koordinatama prosleđenim kao parametri.

```
void lcdDisplayUchar(unsigned char row, unsigned char col,
                    unsigned char value) {
    unsigned char ones;
    unsigned char tens;
    tens = 0x00;
    ones = value;
    while (ones > 9) {
        ones = ones - 10;
        tens++;
    }
    Lcd_Chr(row, col, tens + '0');
    Lcd_Chr(row, col + 1, ones + '0');
}
```

Funkcija lcdDisplayBit Namenska funkcija za prikazivanje bita statusa uzduž drugog reda ekrana.

```
void lcdDisplayBit(unsigned char mask) {
    int i = 0;

    for (i = 0; i <= 15; i++) {
        if ((Status1[i] & mask) == mask) {
            Lcd_Chr(2, 16 - i, '1');
        } else {
            Lcd_Chr(2, 16 - i, '0');
        }
    }
}
```

Funkcija lcdDisplayProgram

Obrada prikaza programa obavlja se u ovoj funkciji. Funkciji se prosleđuje celokupan struct programa kojeg želimo da prikažemo na ekran. Funkcija konverzije cifara za prikaz odrađena je u prethodnim namenskim funkcijama, pa je način prikazivanja jasan i koncizan.

```
void lcdDisplayProgram(struct Mode Program, unsigned char ID) {
    // Lcd_Cmd(_LCD_CLEAR);
    Lcd_Out(1, 1, "Prog:");
    lcdDisplayUchar(1, 6, ID);
    Lcd_Out(1, 9, " ");
    Lcd_Out(1, 13, "Totl"); // prvi red
    //=====drugi red=====
    lcdDisplayUchar(2, 1, Program.startHour);
    Lcd_Chr(2, 3, ':');
    lcdDisplayUchar(2, 4, Program.startMin);
}
```

```

    Lcd_Out(2, 6, "  ");
    Lcd_Chr(2, 8, '/');
    lcdDisplayUchar(2, 9, Program.durationsH);
    lcdDisplayUchar(2, 11, Program.durationsL);
    Lcd_Out(2, 13, "  ");
}

```

Funkcija updateLCD

Celokupna obrada svih 20 prikaza obavljena je u ovoj funkciji. Vršiti se provjera brojača koji je vezan za taster na ploči, nakon koje se bira jedan od dvadeset mogućih prikaza. Prvih šestnaest prikazuju program, njegov broj i njegove podatke. Dok ostali prikazuju statusne bitove Slev automata u vidu bitskog vektora.

```

void updateLCD() {
    if (cntDisp <= 16) {
        lcdDisplayProgram(Program[cntDisp], cntDisp);
    } else if (cntDisp == 17) { // System
        Lcd_Out(1, 1, "Operation      ");
        lcdDisplayBit(STATUS_SYSTEM_BIT);
    } else if (cntDisp == 18) { // Watering
        Lcd_Out(1, 1, "Watering      ");
        lcdDisplayBit(STATUS_WATER_BIT);
    } else if (cntDisp == 19) { // Alarm
        Lcd_Out(1, 1, "Alarm      ");
        lcdDisplayBit(STATUS_ALARM_BIT);
    } else if (cntDisp == 20) { // Manual
        Lcd_Out(1, 1, "Manual      ");
        lcdDisplayBit(STATUS_MANUAL_BIT);
    }
}
}

```

Funkcija main

Glavna funkcija u kojoj se najprije pozivaju funkcije za inicijalizaciju, nakon kojih sledi glavna beskonačna petlja iz koje se izlazi isključivo po prekidu tajmera, tj. na svakih 25ms. U beskonačnoj petlji vršimo prozivku svih Slev automata na svakih 125ms, od kojih kada se prozovu svi, prođe dve sekunde. Prolazak kroz sve bašte označava Flag1. Najprije se šalje broadcast RTC signal (ako je FlagRTC postavljen sa CRS-a), svim Slev automatima u isto vreme, kako bi se izbeglo kašnjenje i zadržala tačnost sata realnog vremena. Nakon slanja podešavanja RTC-a, vreme je da se Slev automati isprogramiraju. Bira se bašta putem browsera kojoj se, kada ona dođe na red, šalju svi potrebni podaci o programu. Posle prolaska kroz međukomunikaciju, LCD ekran se osvežava i započinjemo novu iteraciju petlje.

```

void main(void) {
    init();
    init_variables();

    while (1) {
        SPI_Ethernet_doPacket();
        if (Flag1 == 0x01) { // 125ms

```

```

Flag1 = 0x00;
SLAVE_ID++; // sledeca basta
if (SLAVE_ID == 0x10) { // svi slejovi pollovani
    SLAVE_ID = 0x00;
    PORTA.F4 = 1;
} else
    PORTA.F4 = 0;

if (Garden[SLAVE_ID].gardenSend == 0x01) {
    DR = 1;
    transmit(MODE_CODE | SLAVE_ID); // prvi komandni bajt
    // BrojBaste.BrojPrograma.Start 1Byte
    transmit(Program[Garden[SLAVE_ID].modeID].startHour);
    transmit(Program[Garden[SLAVE_ID].modeID].startMin);
    transmit(Program[Garden[SLAVE_ID].modeID].durationsH);
    transmit(Program[Garden[SLAVE_ID].modeID].durationsL);
    DR = 0;
    Garden[SLAVE_ID].gardenSend = 0x00;
    ByteID = BYTE_ID_CMD_BYTE;

} else if (FlagRTC == 0x01) {
    // saljemo RTC signal svim bastama, da bi smo izbegli kasnjenja
    DR = 1;
    transmit(RTC_BROADCAST); // 0x1F
    // transmit(RTC_CODE|SLAVE_ID);
    transmit(hours);
    transmit(minutes);
    transmit(seconds);
    DR = 0;
    FlagRTC = 0x00;
    ByteID = BYTE_ID_CMD_BYTE;

} else {
    DR = 1;
    transmit(STATUS_CODE | SLAVE_ID); // pitamo za status slejva
    DR = 0;
    ByteID = BYTE_ID_CMD_BYTE; // ocekujemo cmd_byte
}

} // od if (Flag1 == 0x01)

if (updateLCDFlag == 1) {
    updateLCDFlag = 0;
    updateLCD();
}
// od while(1)
}

```


7 Realizacija firmvera Slejv automata

Firmver Slejv automata napisan je za mikroC kompajler i kontroler PIC16F877A. Osnovna uloga firmvera je da lokalno upravlja procesom zalivanja i da Master automatu u svakom trenutku može da vrati ažuran status sistema. Kod za firmver se sastoji od funkcija za inicijalizaciju registara mikrokontrolera, inicijalizaciju globalnih promenljivih, obrade prekida, obrade tastera, komunikacije sa Master automatom, formiranja statusnog bajta, ispisa na LCD i glavne funkcije. U nastavku su date tabele makroa, pinova, promenljivih i funkcija, a zatim i objašnjenja najvažnijih funkcionalnih celina.

7.1 Definicija pinova i deklaracija promenljivih

Na početku programa definisane su simboličke oznake za ulaze i izlaze koji se koriste u kodu. Time se izbegava direktno korišćenje brojeva pinova u ostatku programa i olakšava se povezivanje opisa firmvera sa električnom šemom razvojne ploče.

Oznaka u kodu	Pin	Namena
<code>PinTaster</code>	<code>PORTB.F0</code>	Manuelno pokretanje ili zaustavljanje zalivanja
<code>PinReset</code>	<code>PORTB.F2</code>	Resetovanje lokalnog stanja automata
<code>PinSystemOn</code>	<code>PORTA.F2</code>	Izlazna indikacija da je sistem uključen
<code>PinWatering</code>	<code>PORTA.F3</code>	Izlazna indikacija rada pumpe
<code>PinAlarm</code>	<code>PORTA.F4</code>	Izlazna indikacija alarma protoka
<code>DR</code>	<code>PORTC.F5</code>	Upravljanje smerom RS485 drajvera
<code>DEBOUNCE_TICKS</code>	10	Broj tajmerskih intervala za zaštitu od treperenja tastera

Figure 7.1: Tabela makroa i pinova Slejv firmvera

Data je tabela najvažnijih globalnih promenljivih korišćenih u Slejv firmveru.

Promenljiva	Tip	Opis
GARDEN_ID	unsigned char	ID broj Slejv automata, očitava se sa PORTD.F0-F3
ByteID	unsigned char	Stanje prijema poruke preko UART-a
Hours, Minutes, Seconds	unsigned char	Trenutno vreme dobijeno dekodovanjem BCD cifara
ProgStartHour, ProgStartMin	unsigned char	Vreme početka automatskog zalivanja
Tmp_ProgStartHour, Tmp_ProgStartMin	unsigned char	Privremeno primljene vrednosti vremena početka programa
time_left_high, time_left_low	unsigned char	Primljene cifre trajanja zalivanja
WateringSec	unsigned int	Preostalo vreme zalivanja u sekundama
FlowValue	unsigned char	Trenutno očitana vrednost senzora protoka
FlowMin, FlowMax	unsigned char	Minimalna i maksimalna dozvoljena vrednost protoka
ManualMode	bit	Označava da je zalivanje pokrenuto manuelno
CallFlag	bit	Označava da je stigla prozivka od Master automata
RTCSetupFlag	bit	Označava da treba prepisati novo vreme u interni sat
ProgramSetupFlag	bit	Označava da treba prihvatiti novi program zalivanja
UpdateLCDFlag	bit	Označava periodično osvežavanje LCD displeja i logike zalivanja
m_bSystemOn, m_bWatering, m_bAlarm	bit	Interna stanja koja se pakuju u statusni bajt

Figure 7.2: Tabela globalnih promenljivih Slejv firmvera

7.2 Korišćene funkcije

Ispod je tabela svih funkcija koje nose rad Slejv automata. Kao i kod Master firmvera, posle tabele su izdvojene najvažnije funkcije i delovi koda.

Funkcija	Povratna vrednost	Opis
<code>init()</code>	<code>void</code>	Podešava smer portova, A/D konvertor, Tajmer 1, UART i prekide
<code>init_variables()</code>	<code>void</code>	Postavlja početne vrednosti globalnih promenljivih i čisti LCD
<code>interrupt()</code>	<code>void</code>	Obraduje prekid Tajmera 1 i prijem bajtova preko UART-a
<code>ProcessInputs()</code>	<code>void</code>	Obraduje tastere za manuelno zalivanje i reset sa debounce zaštitom
<code>IncrementTime()</code>	<code>void</code>	Uvećava interni sat za jednu sekundu
<code>DecodeTime()</code>	<code>void</code>	Pretvara BCD cifre sata u promenljive Hours , Minutes i Seconds
<code>ReadADC()</code>	<code>unsigned char</code>	Pokreće A/D konverziju i vraća vrednost protoka iz registra ADRESH
<code>buildStatusByte()</code>	<code>unsigned char</code>	Formira statusni bajt koji Slejv šalje Masteru
<code>toBcd()</code>	<code>unsigned char</code>	Konvertuje decimalnu vrednost u BCD format za poređenje vremena
<code>ConvertTime()</code>	<code>void</code>	Razdvaja primljenu decimalnu vrednost vremena na desetice i jedinice
<code>LcdOut2()</code> , <code>LcdOut3()</code>	<code>void</code>	Formatiraju dvocifrene i trocifrene vrednosti za LCD
<code>LcdOutDuration()</code>	<code>void</code>	Prikazuje preostalo vreme zalivanja u formatu MM:SS
<code>UpdateLCD()</code>	<code>void</code>	Ispisuje trenutno vreme, protok, alarme, program i preostalo vreme zalivanja
<code>transmit()</code>	<code>void</code>	Šalje jedan bajt preko UART predajnika
<code>main()</code>	<code>void</code>	Glavna funkcija programa u kojoj se obrađuju događaji i upravlja pumpom

Figure 7.3: Tabela funkcija Slejv firmvera

7.3 Detaljan opis najvažnijih funkcija sa segmentima koda

Funkcija `init_variables` služi za inicijalizaciju varijabli Slevj automata na po-drazumevane vrednosti. U početnom stanju brišu se flegovi za komunikaciju i tastere, vreme se postavlja na nulu, pumpa i alarm se gase, a sistemska indikacija se uključuje.

```
void init_variables()
{
    GARDEN_ID = 0x00;
    ByteID = 0x00;
    ch = 0x00;
    Command = 0x00;
    Counter = 0x00;
    Counter2 = 0x00;
    cntManual = 0x00;
    cntReset = 0x00;

    CallFlag = 0;
    RTCSetupFlag = 0;
    UpdateLCDFlag = 0;
    ProgramSetupFlag = 0;
    ManualMode = 0;
    ManualEvent = 0;
    ResetEvent = 0;

    WateringSec = 0;
    ProgStartHour = 0x00;
    ProgStartMin = 0x00;
    PinSystemOn = 1;
    PinWatering = 0;
    PinAlarm = 0;

    m_bSystemOn = 0;
    m_bWatering = 0;
    m_bAlarm = 0;
}
```

Funkcija `init` podešava hardver Slevj automata. Port A se delimično koristi za analogni ulaz senzora protoka, a delimično za izlazne indikacije. Port B je podešen kao ulaz za tastere, port C sadrži UART linije, LCD kontrolne linije i DR signal RS485 drajvera, dok se port D koristi i za ID uređaja i za podatke LCD displeja.

A/D konvertor je uključen preko registara `ADCON0` i `ADCON1`. Rezultat konverzije se koristi kao osmobaritna vrednost protoka, tako što se čita registar `ADRESH`. Tajmer 1 se podešava tako da prekid nastaje na svakih 100 ms. Početne vrednosti `TMR1H` = 0x0B i `TMR1L` = 0xDC odgovaraju intervalu od 100 ms za takt mikrokontrolera od 20 MHz i preskaler 1:8. UART se inicijalizuje na 19200 bps, a zatim se dozvoljavaju prekidi Tajmera 1 i prijema preko UART-a.

```
void init()
{
```

```

TRISA = 0x03;
TRISB = 0x3F;
TRISC = 0xC0;
TRISD = 0x0F;

ADCON1 = 0b00001110;
ADCON0 = 0b10000001;

T1CON = 0b00110000;
TMR1H = 0x0B;
TMR1L = 0xDC;
T1CON.TMR1ON = 1;

PIR1.TMR1IF = 0;
PIE1.TMR1IE = 1;

Uart1_Init(19200);
TXSTA.TXEN = 1;
RCSTA.SPEN = 1;
RCSTA.CREN = 1;
PIE1.RCIE = 1;

INTCON.GIE = 1;
}

```

Funkcija ProcessInputs obrađuje taster za manuelno zalivanje i taster za resetovanje lokalnog stanja. Oba tastera imaju debounce zaštitu preko brojača koji se smanjuju na svakom prekidu Tajmera 1. Kada se registruje validan pritisak, funkcija ne izvršava kompletnu akciju odmah, već postavlja odgovarajući događaj koji kasnije obrađuje glavna petlja.

```

void ProcessInputs()
{
    if (cntManual > 0)
        cntManual--;
    if (PinTaster == 0)
        TMP_Taster1 = 0;
    if ((cntManual == 0) && (TMP_Taster1 == 0) && (PinTaster == 1))
    {
        TMP_Taster1 = 1;
        cntManual = DEBOUNCE_TICKS;
        ManualMode = !ManualMode;
        ManualEvent = 1;
    }

    if (cntReset > 0)
        cntReset--;
    if (PinReset == 0)
        TMP_Reset1 = 0;
}

```

```

    if ((cntReset == 0) && (TMP_Reset1 == 0) && (PinReset == 1))
    {
        TMP_Reset1 = 1;
        cntReset = DEBOUNCE_TICKS;
        ResetEvent = 1;
    }
}

```

Funkcija buildStatusByte formira statusni bajt koji Slejv vraća Master automatu. Redosled bita odgovara protokolu opisanom u poglavlju komunikacije: S, W, A i M zauzimaju četiri najviša bita statusnog bajta.

```

unsigned char buildStatusByte()
{
    unsigned char status = 0x00;
    if (m_bSystemOn)
        status |= STATUS_SYSTEM_BIT;
    if (m_bWatering)
        status |= STATUS_WATER_BIT;
    if (m_bAlarm)
        status |= STATUS_ALARM_BIT;
    if (ManualMode)
        status |= STATUS_MANUAL_BIT;
    return status;
}

```

Funkcija interrupt ima dve glavne grane. Prva obrađuje prekid Tajmera 1. Pošto tajmer generiše prekid na svakih 100 ms, brojač **Counter** se koristi da se posle deset intervala dobije period od jedne sekunde. Tada se pozivaju **IncrementTime()** i **DecodeTime()**, a **UpdateLCDFlag** se postavlja na jedinicu. U svakom tajmerskom prekidu poziva se i **ProcessInputs()**, čime se tasteri očitavaju dovoljno često da se pouzdano detektuje pritisak.

Druga grana prekida obrađuje UART prijem. Primljeni bajt se čita iz registra **RCREG**, a promenljiva **ByteID** određuje da li se očekuje prvi komandni bajt ili neki od bajtova korisnog sadržaja. Ako prvi bajt odgovara adresi trenutnog Slejva i tipu komande, postavlja se odgovarajući fleg:

- **CallFlag** za statusnu prozivku,
- **RTCSetupFlag** za podešavanje sata,
- **ProgramSetupFlag** za prihvatanje novog programa zalivanja.

Na ovaj način se u prekidu radi samo prijem i osnovno dekodovanje poruke, dok se duža obrada prebacuje u glavnu petlju.

```

void interrupt()
{
    GARDEN_ID = PORTD & 0x0F;

    if ((PIE1.TMR1IE) && (PIR1.TMR1IF))

```

```

{
    PIR1.TMR1IF = 0;
    if (Counter == 9)
    {
        Counter = 0;
        IncrementTime();
        DecodeTime();
        UpdateLCDFlag = 1;
    }
    else
        Counter++;

    if (Counter2 > 0)
        Counter2--;

    ProcessInputs();
    TMR1H = 0x0B;
    TMR1L = 0xDC;
}

if ((PIE1.RCIE) && (PIR1.RCIF))
{
    ch = RCREG;
    if (ByteID == 0x00)
    {
        if (((ch & 0x0F) == GARDEN_ID) && ((ch & 0xE0) == 0xA0))
        {
            ByteID = 0x08;
            Counter2 = 4;
        }
        else if (ch == 0x7F)
        {
            ByteID = 0x03;
            Counter2 = 3;
        }
        else if (((ch & 0x0F) == GARDEN_ID) && ((ch & 0xE0) == 0x20))
        {
            CallFlag = 1;
        }
    }
}
}

```

Funkcija main najpre poziva inicijalizacione funkcije za registre, promenljive i LCD, a zatim ulazi u beskonačnu petlju. U petlji se proveravaju flegovi koje su postavili prekidi i korisnički ulazi.

Reset događaj zaustavlja zalivanje, briše manuelni režim, gasi alarm i vraća sistem u osnovno uključeno stanje. Manuelni taster radi kao prekidač. Kada se manuelni režim uključi, pumpa se pokreće i u simulacionoj verziji programa dobija

vreme rada od 180 sekundi. Ponovnim pritiskom tastera zalivanje se zaustavlja. Automatsko zalivanje se pokreće kada su ispunjeni sledeći uslovi:

```
m_bWatering = 0,
Seconds = 0,
Hours = ProgStartHour,
Minutes = ProgStartMin.
```

Tada se preostalo vreme zalivanja postavlja na trajanje primljeno od Master automata, aktivira se izlaz pumpe i postavlja se status da je zalivanje u toku. Svake sekunde se WateringSec smanjuje za jedan. Kada dostigne nulu, izlaz pumpe se gasi i Slejv prelazi u stanje čekanja sledećeg programa ili manuelne komande.

```
if (UpdateLCDFlag == 1)
{
    UpdateLCDFlag = 0;

    if ((m_bWatering == 0) &&
        (Seconds == 0x00) &&
        (Hours == ProgStartHour) &&
        (Minutes == ProgStartMin))
    {
        WateringSec = (unsigned int)((time_left_high * 100) + time_left_low);
        m_bWatering = 1;
        PinWatering = 1;
        m_bSystemOn = 1;
        PinSystemOn = 1;
    }

    if (m_bWatering == 1)
    {
        if (WateringSec > 0)
            WateringSec--;
        if (WateringSec == 0)
        {
            m_bWatering = 0;
            PinWatering = 0;
            ManualMode = 0;
        }
    }
}
```

Nadzor protoka i alarm Vrednost senzora protoka očitava se funkcijom ReadADC(). Alarm se proverava samo dok je zalivanje aktivno. Ako je očitana vrednost manja od FlowMin ili veća od FlowMax, postavljaju se PinAlarm i m_bAlarm. U suprotnom se alarm gasi. U implementiranoj verziji početne granice su FlowMin = 20 i FlowMax = 60. Time se detektuju i kvarovi sa premalim protokom, na primer zapušeno crevo ili prazna posuda, i stanja sa prevelikim protokom, na primer pucanje cevi.


```

FlowValue = ReadADC();

if (m_bWatering == 1)
{
    if ((FlowValue < FlowMin) || (FlowValue > FlowMax))
    {
        PinAlarm = 1;
        m_bAlarm = 1;
    }
    else
    {
        PinAlarm = 0;
        m_bAlarm = 0;
    }
}
else
{
    PinAlarm = 0;
    m_bAlarm = 0;
}
}

```

Funkcija UpdateLCD osvežava oba reda displeja. Prvi red prikazuje tekuće vreme i trenutnu vrednost protoka. Drugi red prikazuje alarm, manuelni režim, vreme početka programa i preostalo vreme do gašenja pumpe. Preostalo vreme se prikazuje u formatu minuta i sekundi pomoću funkcije `LcdOutDuration()`.

```

void UpdateLCD()
{
    Lcd_Out(1, 1, "T ");
    Lcd_Chr(1, 3, Hour_X10 + '0');
    Lcd_Chr(1, 4, Hour_X1 + '0');
    Lcd_Chr(1, 5, ':');
    Lcd_Chr(1, 6, Min_X10 + '0');
    Lcd_Chr(1, 7, Min_X1 + '0');
    Lcd_Chr(1, 8, ':');
    Lcd_Chr(1, 9, Sec_X10 + '0');
    Lcd_Chr(1, 10, Sec_X1 + '0');
    Lcd_Out(1, 12, "F");
    LcdOut3(1, 13, FlowValue);

    if (m_bAlarm) Lcd_Chr(2, 1, 'A'); else Lcd_Chr(2, 1, '/');
    if (ManualMode) Lcd_Chr(2, 2, 'M'); else Lcd_Chr(2, 2, '/');

    Lcd_Out(2, 3, "S:");
    Lcd_Chr(2, 4, (Tmp_ProgStartHour / 10) + '0');
    Lcd_Chr(2, 5, (Tmp_ProgStartHour % 10) + '0');
    Lcd_Out(2, 6, ":");
    Lcd_Chr(2, 7, (Tmp_ProgStartMin / 10) + '0');
    Lcd_Chr(2, 8, (Tmp_ProgStartMin % 10) + '0');
}

```

```
Lcd_Out(2, 10, " R");  
LcdOutDuration(2, 12, WateringSec);  
}
```

8 Postupak verifikacije i vrednovanja koda

8.1 Rezultat rada sistema

Verifikacija sistema vrši se povezivanjem Master i Slejv razvojne ploče preko RS-232 veze i učitavanjem odgovarajućih firmvera u mikrokontrolere. ID Slejv automata podešava se prekidačima na donja četiri bita porta D. Nakon uključivanja napajanja proverava se da li se na LCD displeju prikazuje trenutno vreme, vrednost protoka, status alarma i preostalo vreme zalivanja.

Ispravnost komunikacije proverava se periodičnom prozivkom. Master na svakih 125 ms proziva sledeći Slejv i očekuje odgovor sa komandnim i statusnim bajtom. Kada se Slejv odazove, na strani Mastera se ažurira status odgovarajuće bašte, a preko CRS-a se može očitati da li je sistem uključen, da li je zalivanje aktivno, da li postoji alarm i da li je rad pokrenut manuelno.

Automatski rad proverava se slanjem programa zalivanja sa Master automata. Nakon prijema vremena početka i trajanja zalivanja, Slejv čuva program, prikazuje vreme početka na displeju i čeka poklapanje sa internim satom. Kada se vreme poklopi, uključuje se izlaz pumpe, pali se indikacija zalivanja, a preostalo vreme na displeju počinje da se smanjuje. Po isteku zadatog vremena pumpa se automatski isključuje.

Manuelni rad proverava se pritiskom tastera na PORTB.F0. Prvi pritisak pokreće zalivanje i postavlja status manuellnog režima, a drugi pritisak zaustavlja zalivanje. Reset taster na PORTB.F2 vraća Slejv u početno stanje i gasi alarm i pumpu.

Alarm protoka proverava se promenom analogne vrednosti na ulazu senzora protoka. Dok pumpa radi, vrednost ispod minimalne ili iznad maksimalne granice aktivira izlaz alarma i statusni bit alarma u odgovoru Slejva. Kada se vrednost vrati u dozvoljeni opseg, alarm se gasi.

Komunikacija u sistemu proverava se putem centralnog računarskog sistema, gde se na /s zahtev dobijaju povratne informacije o statusu Slejv automata, koji Master automat čuva u svoj buffer. Kada se proverí ispravnost rada Slejv automata kako na lokalnom Master LCD-u, tako i na CRS-u, možemo biti sigurni da je sistem operativan.

8.2 Stepén ispunjenosti korisničkih zahteva

Realizovani Slejv automat ispunjava osnovne zahteve lokalnog uređaja za zalivanje. Uređaj može da prihvati program rada od Master automata, da samostalno pokrene zalivanje u zadato vreme, da isključi pumpu po isteku trajanja programa i da omogući manuelno pokretanje ili zaustavljanje zalivanja. Interni sat može da se podesi daljinski preko Master automata, što omogućava usklađen rad svih Slejv uređaja u sistemu.

Status sistema dostupan je Master automatu kroz periodični odgovor Slejva. Statusni bajt sadrži informacije o uključenosti sistema, aktivnom zalivanju, alarmu i manuelnom režimu, što odgovara zahtevima za periodično očitavanje stanja bašte. Lokalni LCD prikazuje trenutno vreme, vrednost protoka, vreme početka programa i preostalo vreme zalivanja, pa korisnik na samom uređaju može da proverí trenutno stanje.

Nadzor protoka je realizovan preko A/D konvertora i poređenja sa minimalnom i maksimalnom dozvoljenom vrednošću. Time je ispunjen zahtev za signalizaciju neis-

pravnog protoka tokom rada pumpe. U trenutnoj verziji firmvera granice protoka su definisane u programu kao početne vrednosti, dok se daljinsko podešavanje granica može dodati proširenjem protokola na isti način kao što je realizovano podešavanje programa zalivanja.

Realizovani Master automat ispunjava zahteve glavnog kontrolera komunikacije na magistralama Master-Slejev i Master-CRS. CRS komunicira sa Master automatom putem internet browsera koristeći HTTP protokol i GET zahtev, upisivanjem komande direktno u search bar. Master automat čuva podatke o svakom zasebnom programu i svakoj bašti ponaosob. Može da programira Slejev automate UART protokolom, preko RS-232 porta za kratke distance, da preko istog protokola prihvata podatke, čuva ih i direktno prikazuje u internet pretraživač u vidu stringa. Na svom lokalnom LCD-u prikazuje mogućih 16 programa zalivanja zajedno sa njihovim parametrima, kako i njihove statusne poruke.

9 Detaljno uputstvo za upotrebu

Pre korišćenja Slejev automata potrebno je podesiti njegov ID broj pomoću prekidača povezanih na `PORTD.F0-F3`. Svaki Slejev u sistemu mora da ima različit ID u opsegu od 0 do 15. Nakon toga se Slejev povezuje sa Master automatom preko RS-232 konektora i uključuje se napajanje razvojne ploče.

Po uključanju, na LCD displeju Slejva treba da se prikažu trenutno vreme, vrednost protoka i status programa. Ako je potrebno sinhronizovati vreme, sa CRS-a se preko Master automata šalje komanda za podešavanje sata. Master zatim broadcast porukom šalje sat, minut i sekund svim Slejev uređajima.

Program zalivanja se zadaje preko CRS-a izborom programa i dodelom tog programa odgovarajućoj bašti. Master tada šalje Slejvu vreme početka i trajanje zalivanja. Slejev prihvata program, upisuje ga u aktivne promenljive i na LCD displeju prikazuje zadato vreme početka. Kada interni sat dostigne zadato vreme, zalivanje se pokreće automatski.

Manuelno zalivanje pokreće se pritiskom tastera na `PORTB.F0`. Ako zalivanje nije bilo aktivno, pritisak tastera uključuje pumpu i postavlja manuelni režim. Ako je zalivanje već aktivno, sledeći pritisak tastera ga zaustavlja. Reset taster na `PORTB.F2` koristi se kada je potrebno odmah zaustaviti rad pumpe i vratiti lokalne statuse u početno stanje.

Tokom zalivanja treba pratiti LCD displej i indikacione diode. Indikacija zalivanja pokazuje da je pumpa uključena, a alarmna indikacija se uključuje ako je protok izvan dozvoljenog opsega. U tom slučaju potrebno je proveriti dovod vode, pumpu, creva i senzor protoka. Kada se protok vrati u dozvoljene granice, alarm se automatski gasi pri sledećem očitavanju senzora.

10 Appendix

Appendix sadrži sve vrednosti standarda prenosa koje su definisane u deljenom fajlu config.h

```
// serijski prijem (UART) =====
#define UART_BAUD_RATE 19200
// Protokol Master -> Slave:=====
/*
 * =====
 *RTC promenljive 011|x/|idid| -> 0x60 + 3 Bytes HH/MM/|SS
 *Biranje moda 101|x/|idid| -> 0xA0 + 5 Bytes /pXX/HH/MM/SS/SS
 * =====
 */
#define RTC_CODE 0x60
#define RTC_BYTES 3 // DONE
#define MODE_CODE 0xA0
#define MODE_BYTES 5
#define RTC_BROADCAST 0x7F
// Slave -> Master status response byte.
#define STATUS_CODE 0x20
// Byte-id states used while receiving protocol payload bytes.
// 0 means no payload is currently being received.
#define BYTE_ID_IDLE 0x00
#define BYTE_ID_RTC_HOUR 0x04
#define BYTE_ID_RTC_MIN 0x03
#define BYTE_ID_RTC_SEC 0x02
#define BYTE_ID_MODE_PROGRAM 0x09
#define BYTE_ID_MODE_START_HOUR 0x08
#define BYTE_ID_MODE_START_MIN 0x07
#define BYTE_ID_MODE_DURATION_H 0x06
#define BYTE_ID_MODE_DURATION_L 0x05
#define STATUS_SYSTEM_BIT 0x80 // bit7
#define STATUS_WATER_BIT 0x40 // bit6
#define STATUS_ALARM_BIT 0x20 // bit5
#define STATUS_MANUAL_BIT 0x10 // bit4
// MASKE
#define CMD_TYPE_MASK 0xE0 // Top 3 command bits
#define CMD_ID_MASK 0x0F // bottom 4 SLAVE ID bits
#define CMD_INOUT_MASK 0x10 // bit4
// ETHERNET
#define MAC_ADDR {0x00, 0x14, 0xA5, 0x76, 0x19, 0x3f}
#define IP_ADDR {10, 99, 12, 1}
#define HTTP_PORT 80
// OBB MasterRecieve
#define BYTE_ID_IDLE 0x00
#define BYTE_ID_CMD_BYTE 0x02
#define BYTE_ID_STATUS_BYTE 0x01
```

